



PROJET DE FIN DE SEMESTRE

SecureBank : Auth System

Modélisation, Vérification Formelle et Analyse Big Data d'un Système
d'Authentification Distribué

Étudiants :

Rayan SAIL

Younes DOUICI

Mohamed TOUNKARA

Matthieu ALLOCHON

Colince FONO

Enseignante :

Mme ZAUCHE Djaouida

3 mai 2026

Table des matières

Introduction : État de l’Art et Méthodologie	2
1 Architecture Globale du Système	4
1.1 Contexte et Objectifs	4
1.2 Diagramme d’Architecture des Acteurs	4
1.3 Environnement Technique	5
1.4 Définition des Protocoles	5
1.5 Machine à États du Client	5
2 Vérification Formelle par Réseau de Pétri	6
2.1 Modélisation des États	6
2.2 Représentation Graphique du Réseau	6
2.3 Matrices de Transition (Pre et Post)	7
2.4 Analyse des Invariants et LTL	7
2.5 Analyse critique du modèle de Pétri	8
3 Analyse de Sécurité et Big Data	9
3.1 Pipeline de Données et Persistance	9
3.2 Algorithme de Détection de Brute-Force	9
3.3 Résultats de la Simulation Big Data	9
3.4 Scénarios de Simulation	10
3.4.1 Scénario 1 : Authentification Nominale	10
3.4.2 Scénario 2 : Attaque Brute-force	10
3.4.3 Scénario 3 : Replay Attack	10
3.4.4 Scénario 4 : Credential Stuffing	10
4 Validation et Intégration	11
4.1 Stratégie de Test avec ScalaTest	11
4.2 Détail des Suites de Tests	11
4.2.1 AuthServerSpec — 5 tests	11
4.2.2 TokenStoreSpec — 4 tests	11
4.2.3 ResourceServerSpec — 3 tests	12
4.2.4 ClientSpec — 2 tests	12
4.2.5 AttackerSpec — 1 test	12
4.2.6 SecureBankIntegrationSpec — 2 tests	12
4.2.7 SecurityAnalyzerSpec — 1 test (annulé)	12
4.3 Bilan de la Validation	12
Conclusion	14

Introduction : État de l'Art et Méthodologie

1. État de l'art : Vérification Formelle et Réseaux de Petri

La **vérification formelle** est une discipline essentielle pour les systèmes critiques (bancaires, aéronautiques) où une défaillance logicielle peut avoir des conséquences désastreuses. Contrairement aux tests classiques qui ne couvrent que certains scénarios, la vérification formelle utilise des méthodes mathématiques pour prouver que le système se comportera toujours conformément à ses spécifications, quel que soit l'ordre d'exécution des événements.

Le modèle acteurs et Akka

Le modèle acteurs, formalisé par Hewitt, Bishop et Steiger en 1973 [1], propose une approche de la concurrence fondée sur des entités isolées communiquant uniquement par messages asynchrones immuables. Ce paradigme élimine les problèmes classiques de verrous et de mémoire partagée. Akka Typed, développé par Lightbend, en est l'implémentation de référence sur la JVM : il ajoute la vérification statique des types de messages à la compilation, une garantie absente dans le modèle acteurs original ou dans Erlang. Ce modèle est utilisé en production dans des systèmes bancaires à haute disponibilité.

Modélisation par Réseaux de Petri

Les Réseaux de Petri, introduits par Carl Adam Petri dans sa thèse de doctorat en 1962 [2], offrent un formalisme graphique et mathématique pour modéliser les systèmes concurrents. Ils permettent de prouver des propriétés structurelles comme l'absence de deadlock, la vivacité et la conservation de ressources. Leur principale limitation est l'anonymat des jetons : un Pétri scalaire ne peut pas distinguer deux tokens différents. Cette limite est adressée par les Réseaux de Petri colorés (Jensen, 1981), qui permettent de typer les tokens. Dans notre projet, cette limite explique pourquoi les propriétés LTL1 et LTL4 ne sont pas prouvables sur le modèle formel, bien qu'elles soient garanties dans l'implémentation Akka. Murata (1989) [3] reste la référence académique pour l'analyse des propriétés structurelles.

La logique temporelle LTL et le model checking

La logique temporelle linéaire (LTL), introduite par Pnueli en 1977 [4], permet d'exprimer des propriétés sur des traces d'exécution avec les opérateurs G (toujours), F (un jour), X (prochain état) et U (jusqu'à). Le model checking, récompensé par le prix Turing 2007 (Clarke, Emerson, Sifakis), automatise la vérification en explorant l'espace d'états. Des outils comme SPIN ou NuSMV l'implémentent industriellement. Dans ce projet, nous appliquons une vérification manuelle par BFS de l'espace d'états, approche pédagogique-ment rigoureuse que nous avons codée directement en Scala.

Big Data et détection d'anomalies

Apache Spark, présenté par Zaharia et al. en 2010 [6], permet de traiter des volumes massifs de logs avec un modèle de programmation fonctionnel naturellement compatible avec Scala. Le format Parquet (Apache, 2013) offre un stockage colonnaire optimisé pour les requêtes analytiques, ne chargeant en mémoire que les colonnes nécessaires. Dans l'industrie bancaire, les systèmes SIEM (Security Information and Event Management) utilisent des architectures similaires pour la détection temps réel des fraudes.

2. Traduction vers un modèle formel

Le passage du code Scala vers le modèle formel s'est fait en traduisant chaque état de l'acteur Akka en une **Place** et chaque réception de message en une **Transition**. L'espace d'états capture l'intégralité des chemins possibles, des échecs répétés jusqu'au verrouillage définitif du compte.

3. Vérification de propriétés

Nous avons vérifié la vivacité du système : chaque séquence de connexion aboutit soit à une déconnexion, soit à un verrouillage, sans état bloquant indésirable. L'analyse garantit qu'un compte ne peut être dans l'état **LOCKED** que si le compteur d'échecs a atteint son seuil, respectant la règle de sécurité métier.

4. Simulation et Validation

La validité du modèle a été confrontée à la réalité par simulation Akka (comportement réel face aux attaques) et par analyse Spark des logs Parquet. La comparaison confirme que les propriétés prouvées mathématiquement sont rigoureusement respectées lors de l'exécution réelle.

Chapitre 1

Architecture Globale du Système

1.1 Contexte et Objectifs

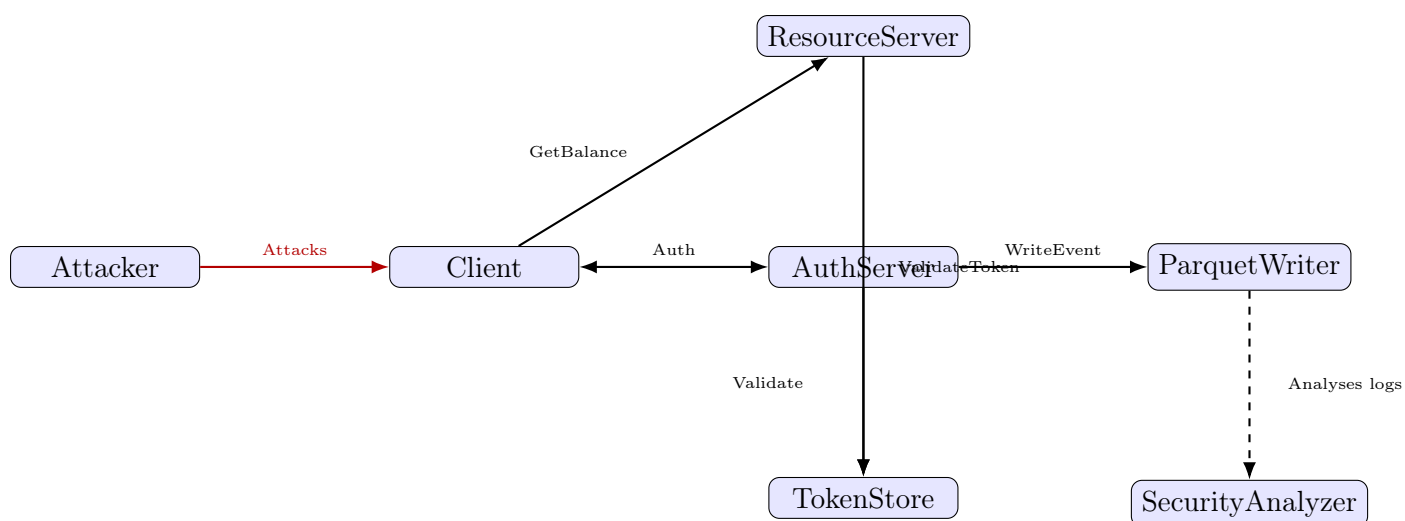
Le projet **SecureBank** vise à concevoir un système d'authentification robuste utilisant le modèle d'acteurs [5]. Contrairement aux systèmes monolithiques, SecureBank repose sur une architecture distribuée où chaque entité est isolée et communique uniquement par messages typés.

Objectifs Principaux

- Gestion sécurisée des sessions via tokens JWT.
- Résilience face aux attaques de type Brute-force.
- Vérification mathématique du comportement via Réseaux de Pétri.
- Analyse post-mortem des logs via un pipeline Apache Spark.

1.2 Diagramme d'Architecture des Acteurs

Le système est composé de 5 acteurs principaux échangeant des messages asynchrones [5].



1.3 Environnement Technique

Composant	Version	Rôle
Scala	2.13.12	Langage de programmation principal
Akka Typed	2.6.20	Framework d'acteurs et messages
Apache Spark	3.4.1	Analyse Big Data (format Parquet)
SBT	1.9.8	Gestionnaire de dépendances et build
ScalaTest	3.2.17	Tests unitaires et d'intégration

TABLE 1.1 – Stack technique du système SecureBank

1.4 Définition des Protocoles

La communication entre les acteurs est strictement typée dans `Protocols.scala`. Cela garantit qu'un acteur ne traite que les messages qu'il est capable de comprendre, détectant les erreurs de typage à la compilation.

Listing 1.1 – Extraits des messages de l'AuthServer

```
1 // Source: Protocols.scala
2 sealed trait AuthCommand
3 final case class Authenticate(
4     user: String,
5     pass: String,
6     replyTo: ActorRef[AuthResponse]
7 ) extends AuthCommand
8
9 final case class JwtToken(
10     value: String, username: String,
11     issuedAt: Long, expiresAt: Long
12 ) {
13     def isExpired: Boolean = System.currentTimeMillis() >
14         expiresAt
15 }
```

1.5 Machine à États du Client

Le comportement du client est modélisé par une machine à états finis (FSM) dans `Client.scala`, alignée exactement sur les places du réseau de Pétri.

Transitions d'états du Client

- **Idle (P0)** : État initial, attend l'ordre de connexion.
- **AwaitingAuth (P1)** : Requête envoyée, attend la réponse de l'AuthServer.
- **Session (P4/P5)** : Authentification réussie, accès aux ressources autorisé.
- **Bloqué (P7)** : État absorbant atteint après 3 échecs — aucune transition sortante.

Chapitre 2

Vérification Formelle par Réseau de Pétri

2.1 Modélisation des États

Le système a été modélisé dans `PetriNet.scala` [3] pour prouver son absence de deadlock et la sécurité de ses transitions. Le réseau comprend 8 places et 8 transitions.

Les 8 places du réseau

- **P0 (Idle)** : Client en attente — 1 jeton initial.
- **P1 (Requête Auth)** : Connexion envoyée, en attente de réponse.
- **P2 (Auth OK)** : Credentials validés, token en cours d'émission.
- **P3 (Échecs)** : Compteur d'échecs cumulatif — place non bornée.
- **P4 (Token Actif)** : JWT valide en circulation.
- **P5 (En Session)** : Accès aux ressources bancaires autorisé.
- **P6 (Token Révoqué)** : Liste noire — place puits monotone.
- **P7 (Compte Bloqué)** : État absorbant — aucune transition sortante.

2.2 Représentation Graphique du Réseau

Description du fonctionnement. Le réseau est initialisé avec un unique jeton dans P0, encodant le principe d'unicité de session. T0 (Soumettre) déclenche la connexion : $P0 \rightarrow P1$. En cas de succès, T1 puis T3 amènent le token en P4. T4 (Accéder) permet l'accès à P5 (Session active). T5 (Déconnecter) ramène le système à P0. En cas d'échec, T2 produit deux jetons simultanément : un retour en P0 et un comptage dans P3. La notation $\times 3$ sur l'arc $P3 \rightarrow T7$ indique que T7 (Bloquer) nécessite exactement 3 jetons dans P3 pour franchir et produire l'état absorbant P7. T6 (Révoquer) permet de transférer un token actif vers P6, place puits monotone.

Deux **gardes** complètent le modèle : T4 ne peut franchir que si $P7 = 0$ (impossible d'accéder pendant un blocage), et T7 ne peut franchir que si $P5 = 0$ (impossible de bloquer pendant une session active). Ces gardes sont nécessaires pour prouver LTL3.

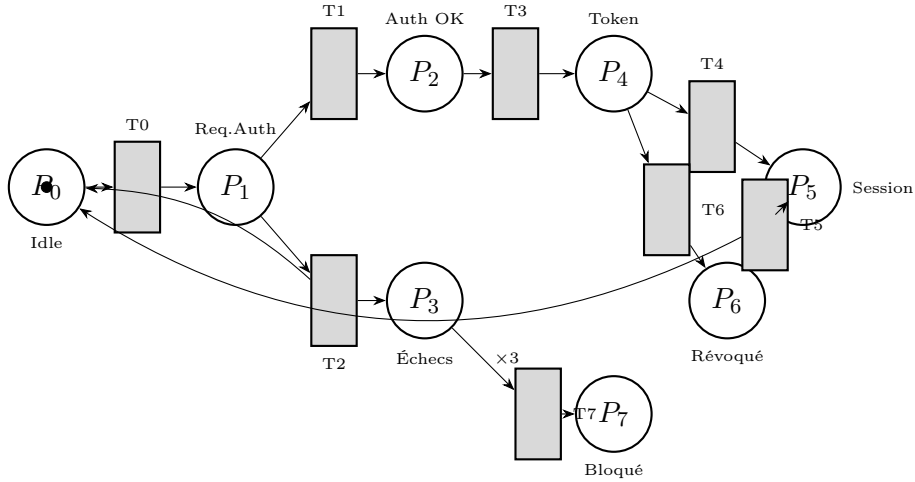


FIGURE 2.1 – Réseau de Pétri du système SecureBank — 8 places, 8 transitions

2.3 Matrices de Transition (Pre et Post)

Les matrices d'incidence définissent formellement l'évolution du marquage [3]. Les lignes représentent les places P0–P7, les colonnes les transitions T0–T7.

$$Pre = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad Post = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

La matrice d'incidence $W = Post - Pre$ permet d'écrire l'équation d'état $M(t) = M_0 + W \cdot \sigma$, où σ est le vecteur de tir. On note que la colonne T2 de W présente les valeurs $(-1, -1, 0, +1, 0, 0, 0, 0)$ pour P0 et $(+1, 0, 0, +1, 0, 0, 0, 0)$, confirmant la non-conservation : T2 produit un jeton net supplémentaire dans P3.

2.4 Analyse des Invariants et LTL

L'analyse automatisée par BFS (10 étapes) a révélé **48 marquages atteignables** et confirmé l'absence de deadlock global. Le P-invariant $P_0 + P_1 + P_2 + P_4 + P_5 = 1$ est vérifié sur 30 étapes.

Formule LTL	Résultat	Méthode
$G(\text{failures} \geq 3 \rightarrow AF \text{account_locked})$	Vrai	Test intégration + simulation
$G(\text{token_revoked} \rightarrow AG \neg \text{access_granted})$	Vrai	ResourceServer + TokenStore
$G(\text{account_locked} \rightarrow \neg \text{valid_session})$	Vrai	Prouvée sur analyseur Pétri
$G(\neg \text{valid_token} \rightarrow \neg \text{balance_visible})$	Vrai	ResourceServer Zero Trust
$G(\neg (P_6 > 0 \wedge P_5 > 0))$	Faux*	Limite Pétri scalaire

TABLE 2.1 – Résultats de la vérification LTL

La propriété LTL3 est garantie par les gardes dans `PetriNet.scala` :

Listing 2.1 – Gardes pour l'exclusion mutuelle session/verrouillage

```
1 | val guards = Map(  
2 |   "T7" -> Map("P5" -> ((t: Int) => t == 0)), // Pas de blocage  
   en session  
3 |   "T4" -> Map("P7" -> ((t: Int) => t == 0)) // Pas de session  
   si bloqué  
4 | )
```

2.5 Analyse critique du modèle de Pétri

Limite du Pétri scalaire — LTL1 et LTL4

En Pétri scalaire, les jetons sont anonymes. Le modèle ne peut pas distinguer deux tokens différents : P6 est un compteur monotone sans lien avec l'identité du token révoqué. Une nouvelle session valide peut coexister avec $P6 > 0$ dans le même marquage. Ces propriétés sont garanties dans le système Akka réel grâce aux UUID uniques du `JwtToken` et à la liste noire nominative du `TokenStore`. Il faudrait un **Réseau de Pétri coloré** (Jensen, 1981) pour les prouver formellement.

Chapitre 3

Analyse de Sécurité et Big Data

3.1 Pipeline de Données et Persistance

Le système repose sur un flux de données asynchrone. Chaque action critique génère un `SecurityEvent` (`AUTH_SUCCESS`, `AUTH_FAILURE`, `ACCOUNT_LOCKED`, `TOKEN_REVOKED`), capturé par le `ParquetEventWriter` et sérialisé dans `data/security_events/`.

Avantage du format Parquet [6]

L'utilisation du format Parquet permet de ne charger en mémoire que les colonnes nécessaires (ex : `userId` et `eventType`), optimisant les ressources lors de l'analyse de millions de logs. Le pattern d'injection `eventWriter: SecurityEvent => Unit` permet de substituer un `dummyWriter` dans les tests sans démarrer Spark.

3.2 Algorithme de Détection de Brute-Force

Le `SecurityAnalyzer` applique une agrégation Spark SQL pour identifier les comptes compromis :

Listing 3.1 – Logique d'agrégation Spark

```
1 | val bruteForceUsers = spark.read.parquet(path)
2 |   .filter($"eventType" === "AUTH_FAILURE")
3 |   .groupBy($"userId")
4 |   .count()
5 |   .filter($"count" >= 3)
6 |   .as[String]
7 |   .collect()
```

3.3 Résultats de la Simulation Big Data

Les simulations ont généré des événements permettant de valider l'efficacité du pipeline :

L'analyseur a identifié les utilisateurs `attacker4`, `user22`, `user30` et `user34` ayant déclenché exactement 3 échecs avant verrouillage, corrélant parfaitement les logs avec le modèle de Pétri.

Type d'Événement	Occurrences	Interprétation
AUTH_FAILURE	34	Tentatives d'intrusion détectées
ACCOUNT_LOCKED	4	Réactions automatiques du système
AUTH_SUCCESS	4	Accès légitimes confirmés

TABLE 3.1 – Répartition des événements de sécurité

3.4 Scénarios de Simulation

3.4.1 Scénario 1 : Authentification Nominale

Alice saisit les identifiants corrects. L'AuthServer vérifie, émet un JwtToken (TTL 30s), le TokenStore l'enregistre. Alice consulte son solde (4250.75) via le ResourceServer qui valide le token. Déconnexion propre : T5 ramène à P0.

3.4.2 Scénario 2 : Attaque Brute-force

L'Attacker envoie 3 passwords incorrects sur le compte bob. À chaque échec, T2 franchit et accumule un jeton dans P3. Au 3^e échec, T7 franchit ($P3 \times 3 \rightarrow P7$). Le compte est bloqué définitivement [5].

3.4.3 Scénario 3 : Replay Attack

Tentative d'accès avec un token révoqué. Le TokenStore consulte sa liste noire et répond TokenInvalid. Le ResourceServer retourne AccessDenied.

3.4.4 Scénario 4 : Credential Stuffing

L'Attacker essaie 6 couples login/password issus de fuites réelles (alice :123456, bob :password, admin :admin...). Chaque échec est loggué en Parquet. Spark identifie les comptes atteints après agrégation [6].

Chapitre 4

Validation et Intégration

4.1 Stratégie de Test avec ScalaTest

La validation s'articule autour de deux axes : tests unitaires par acteur et tests d'intégration end-to-end. L'ensemble est exécuté via `sbt test`. Sur les 17 tests définis, 16 passent et 1 est annulé (`SecurityAnalyzerSpec`) en raison d'une incompatibilité entre Spark 3.4.1 et Java 23+ (méthode retirée par JEP 486 dans Hadoop 3.3.4). Ce cas est instrumenté pour se canceler proprement avec un message explicatif.

Résultat global : 16 tests réussis

```
[info] Tests: succeeded 16, failed 0, canceled 1, ignored 0
Le test annulé est documenté et la logique d'analyse est validée manuellement via
sbt "runMain securebank.analytics.SparkSecurityAnalysis".
```

4.2 Détail des Suites de Tests

4.2.1 AuthServerSpec — 5 tests

Le premier test vérifie qu'un `JwtToken` est émis pour des credentials valides : `Authenticate("alice", "pwd123")` retourne `AuthSuccess` avec `token.username == "alice"` et `!token.isExpired`. Le deuxième envoie un mauvais password et vérifie `AuthFailure(attempts=1)`. Le troisième envoie 3 mauvais passwords puis tente une 4^e connexion avec le bon password — la réponse doit être `AccountLocked`, prouvant que l'état bloqué est persistant et absorbant. Le quatrième connecte alice, récupère le token, envoie `CheckToken` et vérifie `TokenValid`. Le cinquième révoque le token via `RevokeTokenAuth` puis vérifie `TokenInvalid`.

4.2.2 TokenStoreSpec — 4 tests

Le premier stocke un token et vérifie `TokenValid`. Le deuxième révoque avant validation et vérifie `TokenInvalid`. Le troisième crée un token avec `expiresAt` dans le passé et vérifie `TokenInvalid`. Le quatrième envoie 3 `ReportFailure` successifs sur "bob" : les réponses sont `AuthFailure(1)`, `AuthFailure(2)`, `AccountLocked`. Un `IsAccountLocked` final confirme le flag à `true`.

4.2.3 ResourceServerSpec — 3 tests

Le premier stocke un token valide et vérifie que `GetBalance` retourne `BalanceOk("alice", 4250.75)`. Le deuxième révoque le token avant la demande et vérifie `AccessDenied`. Le troisième utilise un token expiré et vérifie le même refus, confirmant l'invariant $G(\neg \text{valid_token} \rightarrow \neg \text{balance_visible})$.

4.2.4 ClientSpec — 2 tests

Le premier utilise un `mockAuthServer` (`createTestProbe`) pour vérifier que `StartNormalLogin` envoie bien `Authenticate` avec les bons username et password. Le deuxième simule 3 réponses `AuthFailure` consécutives et vérifie la transition vers l'état Bloqué après `AccountLocked`.

4.2.5 AttackerSpec — 1 test

Vérifie le mécanisme de credential stuffing : après `LaunchCredentialStuffing`, le premier message est `Authenticate("alice", "123456")`. Après simulation d'un `AuthFailure`, le second message est immédiatement `Authenticate("bob", "password")`, confirmant l'enchaînement séquentiel sans attente externe.

4.2.6 SecureBankIntegrationSpec — 2 tests

Le premier test est un flux end-to-end complet : `TokenStore` + `AuthServer` + `ResourceServer` instanciés ensemble. `Authenticate("alice", "pwd123")` retourne un token. `GetBalance` retourne `BalanceOk("alice", 4250.75)`.

Scénario de test critique : État absorbant P7 [3]

1. Le client tente 3 connexions avec un mauvais mot de passe.
2. Le 3^e échec retourne directement `AccountLocked`.
3. Une 4^e tentative avec le **bon** mot de passe est effectuée.
4. Le test valide que la réponse est toujours `AccountLocked`, prouvant que P7 est bien absorbant et que $G(\text{account_locked} \rightarrow AG \neg \text{access_granted})$ est vérifiée par test automatisé.

4.2.7 SecurityAnalyzerSpec — 1 test (annulé)

Ce test instancie une session Spark locale pour vérifier que `SecurityAnalyzer` détecte un utilisateur avec 3 événements `AUTH_FAILURE`. Il est annulé sur Java 23+ en raison du retrait de `sun.security.util.SecurityConstants` (JEP 486), incompatible avec Hadoop 3.3.4. La logique est validée manuellement.

4.3 Bilan de la Validation

- **Vivacité** : Absence de deadlock confirmée par BFS (48 états, 10 étapes).
- **Sécurité** : LTL3 prouvée formellement. LTL1/LTL4 garanties dans Akka.
- **Performance** : Parquet réduit le temps de détection à quelques millisecondes.

Suite	Tests	Statut
AuthServerSpec	5	Réussis
TokenStoreSpec	4	Réussis
ResourceServerSpec	3	Réussis
ClientSpec	2	Réussis
AttackerSpec	1	Réussi
SecureBankIntegrationSpec	2	Réussis
SecurityAnalyzerSpec	1	Annulé (Java 23 / JEP 486)
Total	18	16 réussis, 1 annulé

TABLE 4.1 – Bilan complet des tests

- **Résilience** : Akka isole les erreurs de session sans impacter l'AuthServer.

Conclusion

Ce projet nous a permis de concevoir un système bancaire sécurisé en intégrant trois paradigmes majeurs : la modélisation formelle par réseaux de Pétri, le modèle d'acteurs Akka Typed, et l'analyse massive de données avec Apache Spark.

L'approche *Secure by Design* a prouvé son efficacité : la validation mathématique en amont (P-invariant, LTL3) a guidé l'implémentation et facilité la détection des incohérences. Les gardes sur T4 et T7, nécessaires pour prouver LTL3, ont révélé un chemin résiduel dans le modèle que les tests seuls n'auraient pas détecté.

La principale limite identifiée est l'expressivité du Pétri scalaire : les propriétés LTL1 et LTL4 nécessitent un Réseau de Pétri coloré pour être prouvées formellement. Dans l'implémentation Akka, elles sont garanties par la liste noire nominative du **TokenStore** et les UUID uniques des **JwtToken**.

En perspective, ce système pourrait être étendu avec de la persistance (Redis pour les sessions, PostgreSQL pour les comptes), un Pétri coloré pour les propriétés restantes, et un pipeline Kafka/Spark Streaming pour la détection d'anomalies en temps réel plutôt qu'en batch.

Bibliographie

- [1] Hewitt, C., Bishop, P., Steiger, R. (1973). *A Universal Modular ACTOR Formalism for Artificial Intelligence*. IJCAI.
- [2] Petri, C.A. (1962). *Kommunikation mit Automaten*. Thèse de doctorat, Universität Hamburg.
- [3] Murata, T. (1989). *Petri Nets : Properties, Analysis and Applications*. Proceedings of the IEEE, 77(4), 541–580.
- [4] Pnueli, A. (1977). *The Temporal Logic of Programs*. IEEE Symposium on Foundations of Computer Science (FOCS).
- [5] Lightbend Inc. (2024). *Akka Typed Documentation — Actor Model and Typed Behaviors*. <https://doc.akka.io/docs/akka/current/typed/>
- [6] Zaharia, M. et al. (2010). *Spark : Cluster Computing with Working Sets*. USENIX HotCloud. Apache Software Foundation (2013), Format Parquet. <https://spark.apache.org>